# ISM8 – Architecture d'un code de dynamique moléculaire

1 – Réduire la complexité d'un terme de lennard jones

2 – Terme de Lennard Jones et OPENMP

3 – Vectorisation

4 – Recherche des plus proches voisins, algorithme en *O(N)*

5 – MPI et plus proches voisins

# ISM8 – TERME DE LENNARD JONES DE COMPLÉXITÉ $O(N^2)$

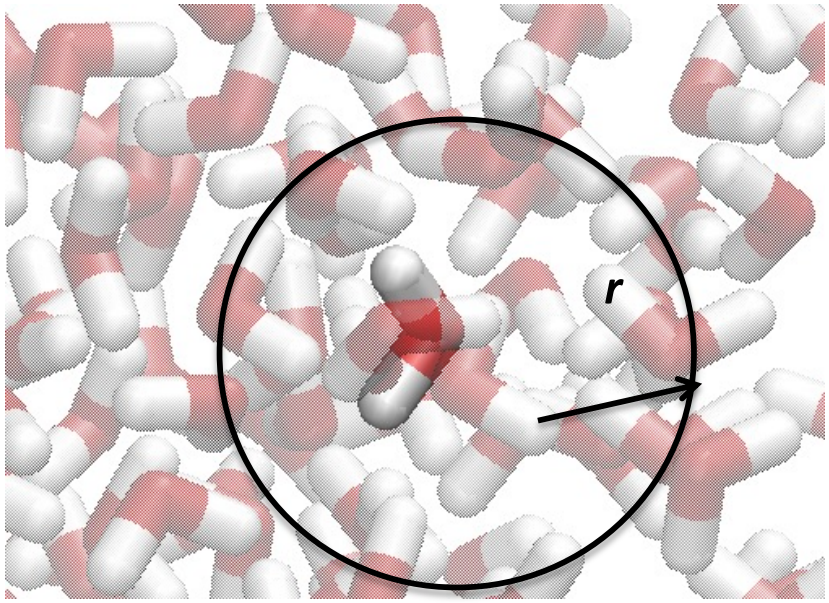
$$U^{LJ} = \sum_{i=1}^{N} \sum_{j=i+1}^{N} \varepsilon_{ij} \left[ \left( \frac{r_{ij}^*}{r_{ij}} \right)^{12} - \left( \frac{r_{ij}^*}{r_{ij}} \right)^{6} \right]$$

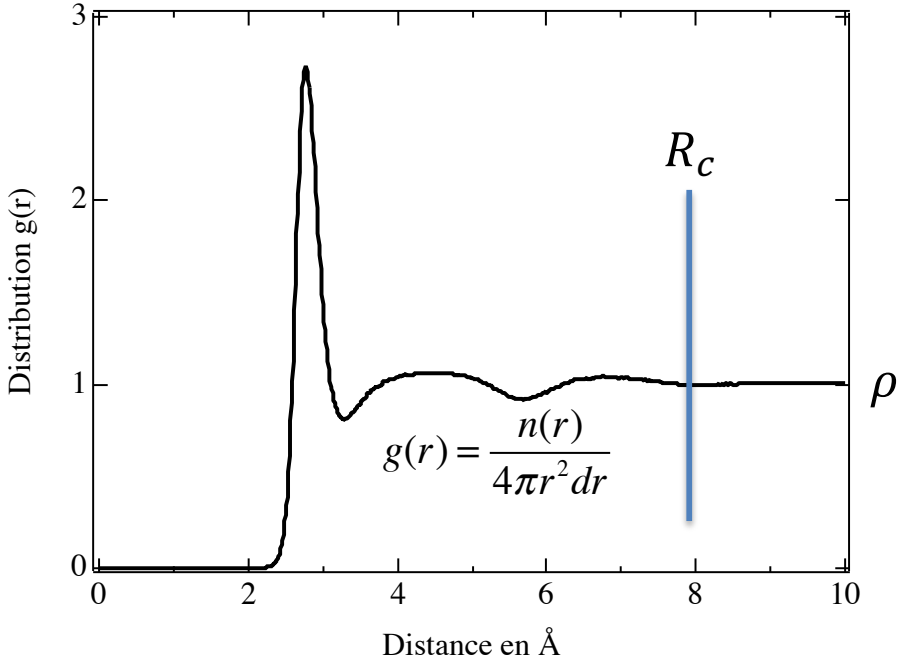
Algorithmiquement, le code le plus « naïf » pour le calculer est de la forme :

```
DO i = 1,N-1
  DO j = i+1,N
      R_ij^2 = (x(i)-x(j))^2 +  (y(i)-y(j))^2 + (z(i)-z(j))^2
      Calcul des éléments de U^LJ
      Calcul du gradient analytique de U^LJ (donc des forces associées) pour
      chacun des atomes i et j
   END DO
END DO
```

**Approche non efficace, tout le temps cpu est employé pour calculer un terme dont la moyenne est constante…**

# ISM8 – RAYON DE COUPURE ET ENERGIE




Si le rayon de coupure $R_c$ est pris tel que $g(r > R_c) = Cte = \rho$, alors l'erreur commise sur l'énergie potentielle totale d'un atome particulier est :

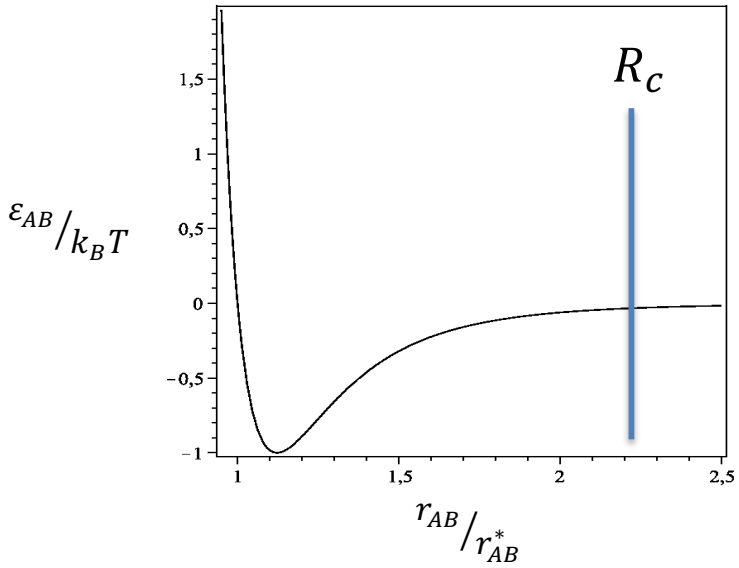
$$\Delta U(R_c) = \int_{R_c}^{+\infty} \frac{A}{r^n} \times g(r) \times 4\pi r^2 dr = 4\pi\rho \int_{R_c}^{+\infty} \frac{A}{r^{n-2}}\, dr$$

Cette erreur est finie si et seulement si $n > 3$ , et dans ce cas

$$\Delta U(R_c) = \left[ -\frac{4\pi A\rho}{(n-3)r^{n-3}} \right]_{R_c}^{+\infty} = \frac{4\pi A\rho}{(n-3)R_c^{n-3}}$$

# ISM8 –TERME DE LENNARD JONES ET REDUCTION DE COMPLEXITÉ

La convergence ‘rapide’ vers zéro du terme de Lennard Jones permet d’employer un rayon de coupure et donc de réduire ***a priori*** la complexité de son calcul de $O(N^2)$ à $O(N)$ ….



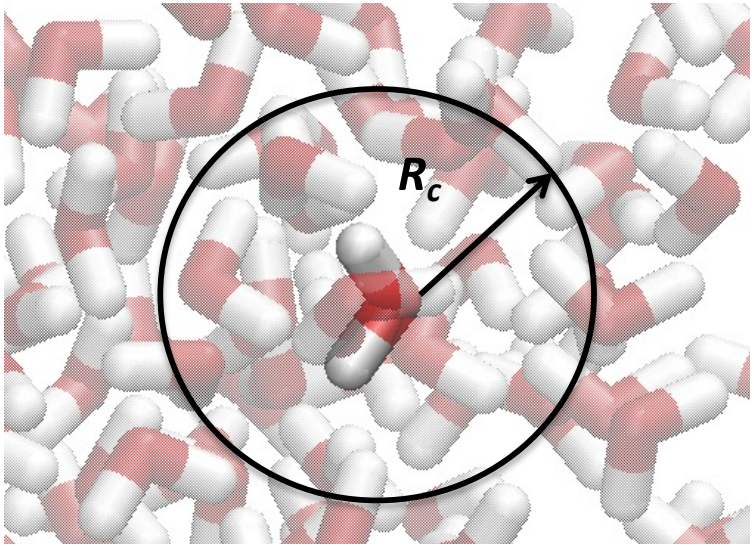

$$U^{LJ} = 4\sum_{i=1}^{N}\sum_{j>i}^{N}\varepsilon_{ij}^{*}\left[\left(\frac{r_{ij}^{*}}{r_{ij}}\right)^{12} - 2\left(\frac{r_{ij}^{*}}{r_{ij}}\right)^{6}\right] \quad O(N^2)$$

$$U^{LJ} \approx 4\sum_{i=1}^{N}\sum_{j/r_{ij}<R_c}\varepsilon_{ij}^{*}\left[\left(\frac{r_{ij}^{*}}{r_{ij}}\right)^{12} - 2\left(\frac{r_{ij}^{*}}{r_{ij}}\right)^{6}\right] \quad O(N)$$

Mais une complexité en *O(N)* sous entend ici que l’on peut identifier les paires d’atomes *(i,j)* qui vérifient $r_{ij} < R_c$ avec un algorithme lui même en *O(N)*, **sinon la recherche des plus proches voisins va imposer sa complexité algorithmique à l’ensemble du code.**

# ISM8 – LISTE DE VERLET

Par définition : liste des particules/atomes situés « à proximité » de chaque particule/atome (liste des plus proches voisins)




On peut bâtir ce type de liste en calculant toutes les distances :

```
DO i = 1,N-1
  DO j = i+1,N
     Rij2 = (x(i)-x(j))2 +  (y(i)-y(j))2 + (z(i)-z(j))2
     if (Rij2 < Rc*Rc) mise en mémoire de la paire
  END DO
END DO
```

**=> algorithme *O(N²)***

L'emploi d'un rayon de coupure $R_c$ transforme le terme de Lennard Jones en une fonction discontinue en $R_c$. Pour corriger ce problème il faut transformer le terme de Lennard Jones tronqué en (avec $n_i$ le nombre de proche voisin de l'atome $i$) :

$$U^{LJ} = \sum_{i=1}^{N}\sum_{j=i}^{n_i} \varepsilon_{ij}\left[\left(\frac{r_{ij}^*}{r_{ij}}\right)^{12} - \left(\frac{r_{ij}^*}{r_{ij}}\right)^{6}\right] \times P_5(r_{ij})$$

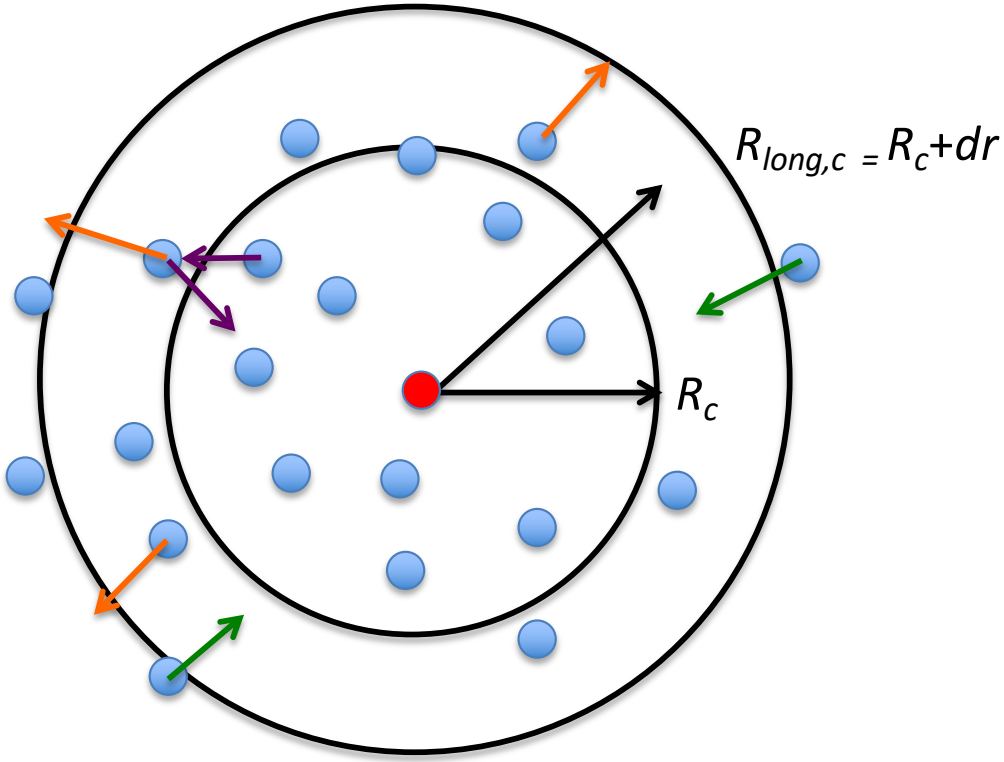
$$P_5(r) = 1 - 10\left(\frac{r - x_{\min}}{r_{\max} - x_{\min}}\right)^3 + 15\left(\frac{r - x_{\min}}{r_{\max} - x_{\min}}\right)^4 - 6\left(\frac{r - x_{\min}}{r_{\max} - x_{\min}}\right)^5$$

$$\text{avec} \quad x_{\min} \le x \le x_{\max} \ \text{et} \ (x_{\min} = R_c - \delta r, x_{\max} = R_c)$$

# ISM8 – Mise à jour des listes de Verlet

Aux conditions ambiantes (température et pression), les molécules d'eau en phase liquide se déplacent d'environ 0,1 Å ($10^{-11}$ m) en 1 fs et « rebondissent » les unes sur les autres.

Il suffit donc de construire des listes de Verlet correspondant à un rayon de coupure $R_c$ plus grand. Lors d'une simulation de type dynamique moléculaire, ces listes de Verlet seront valables sur un intervalle de temps de plusieurs fs. Elles permettent donc de calculer plusieurs fois le terme de Lennard Jones sans avoir à reconstruire systématiquement ces listes.




**Algorithme dont l'efficacité dépend de la quantité de mémoire disponible. Ici on joue sur le pré-facteur et non sur la complexité algorithmique, ici toujours en $O(N^2)$.**

# ISM8 – Terme de Lennard Jones et Listes de Verlet

## Algorithmes pour le calcul de Lennard Jones…

### Sans listes de Verlet ‘étendues’

*m* itérations

```
DO i = 1,N-1
      DO j = i+1,N
          Rij² = (x(i)-x(j))² + (y(i)-y(j))² + (z(i)-z(j))²
          if (Rij² < Rc*Rc) mise en mémoire paire ij
      END DO
END DO

DO i = 1,N
  DO j = 1,ni
      nj = li(j)
      Rij² = (x(i)-x(nj))² + (y(i)-y(nj))² + (z(i)-z(nj))²
                              …
                  Calcul de l’énergie et des forces
  END DO
END DO
```

…

### avec listes ‘étendues’

```
DO i = 1,N-1
      DO j = i+1,N
          Rij² = (x(i)-x(j))² + (y(i)-y(j))² + (z(i)-z(j))²
          if (Rij² < Rlong,c*Rlong,c) mise en mémoire paire ij
      END DO
END DO
```

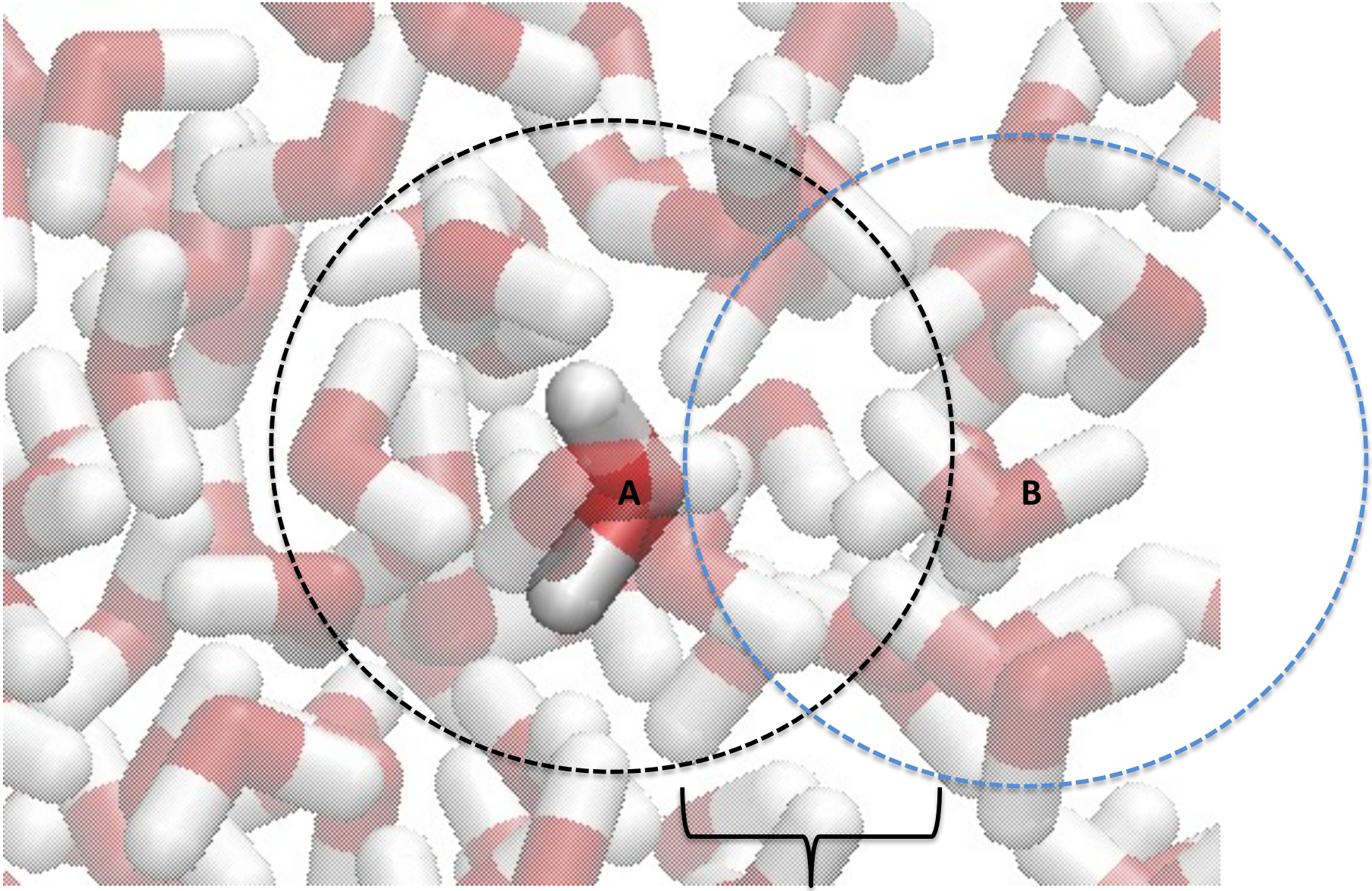
*m* itérations

```
DO i = 1,N
  DO j = 1,ni
      nj = li(j)
      Rij² = (x(i)-x(nj))² + (y(i)-y(nj))² + (z(i)-z(nj))²
      if (Rij² < Rc*Rc)   Pénalité !
                  Calcul de l’énergie et des forces
  END DO
END DO
```

Avec des listes de Verlet étendues, le pré-facteur de la recherche en $O(N^2)$ des proches voisins est divisé par le nombre d’itération *m*, mais il faut encore ajouter un test pour dire au code quand reconstruire les listes de Verlet….

**=> *m* n’est pas constant, mais dans certain cas il peut être très grand (modélisation à basse température, modélisation d’un solide…)**

# ISM8 –TERME DE LENNARD JONES ET MEMORY RACE CONDITION




liste de voisins communs
à **A** et **B**

# ISM8 – TERME DE LENNARD JONES ET OPENMP

Les directives OPENMP s'appliquent sur la première boucle du calcul du terme de Lennard Jones (quelque soit sa complexité)

```
!$OMP REDUCTION(+ : U^LJ)
!$OMP DO
DO i = 1,N
  Fxi = 0 ; Fyi = 0 ; Fzi = 0
  DO j = 1,n_i
     n_j = l_i(j)
     R_ij^2 = (x(i)-x(n_j))^2 +  (y(i)-y(n_j))^2 + (z(i)-z(n_j))^2
     ....
     Calcul de l'énergie U^LJ
     Calcul des forces sur i et calcul des forces sur n_j
      Fxi = Fxi  - ∂U^LJ/∂x_i ; Fx(n_j) = Fx(n_j) - ∂U^LJ/∂x_nj
      Fyi = Fyi  - ∂U^LJ/∂y_i ; Fy(n_j) = Fy(n_j) - ∂U^LJ/∂y_nj
      Fzi = Fzi  - ∂U^LJ/∂z_i ; Fz(n_j) = Fz(n_j) - ∂U^LJ/∂z_nj
   END DO
   Fx(i) = Fxi ; Fy(i) = Fyi ; Fz(i) = Fzi
END DO
!$OMP ENDDO
```

➔ Ici , seulement $n_j > i$ dans $l_i$ *(approche 'triangulaire')*

➔ **« race condition ».**

Le problème de « race condition » a pour origine le fait qu'un même atome peut être un proche voisin de deux atomes distincts…

# ISM8 – FORCES DE LENNARD JONES ET OPENMP

Pour résoudre le problème de race conditions sur les forces différentes possibilités, dont :

```
!$OMP REDUCTION(+ : U^LJ)
!$OMP DO
DO i = 1,N
   ...
  DO j = 1,n_i
      n_j = l_i(j)
      R_ij^2 = (x(i)-x(n_j))^2 +  (y(i)-y(n_j))^2 + (z(i)-z(n_j))^2
      ....
      Calcul de l'énergie U^LJ
      Calcul des forces
!$OMP ATOMIC
      Fx(n_j) = Fx(n_j) - ∂U^LJ/∂x_nj
!$OMP ATOMIC
      Fy(n_j) = Fy(n_j) - ∂U^LJ/∂y_nj
!$OMP ATOMIC
      Fz(n_j) = Fz(n_j) - ∂U^LJ/∂z_nj
   END DO
   ...
END DO
!$OMP ENDDO
```

➔ **Pas efficace (pénalité de 30 à 50%)**

# ISM8 – FORCES DE LENNARD JONES ET OPENMP

Les listes de proches voisins peuvent être construites en prenant tous les voisins, dans ce cas il est inutile de calculer les forces correspondant aux voisins

```
!$OMP REDUCTION(+ : U^LJ)
!$OMP DO
DO i = 1,N
   Fxi = 0 ; Fyi = 0 ; Fzi = 0
  DO j = 1,n_i
      n_j = l_i(j)

      R_ij^2 = (x(i)-x(n_j))^2 +  (y(i)-y(n_j))^2 + (z(i)-z(n_j))^2
      Calcul de l'énergie U^LJ
      Calcul des forces
      Fx(n_j) = Fx(n_j) - ∂U^LJ/∂x_nj        Fxi = Fxi  - ∂U^LJ/∂x_i
      Fy(n_j) = Fy(n_j) - ∂U^LJ/∂y_nj        Fyi = Fyi  - ∂U^LJ/∂y_i
      Fz(n_j) = Fz(n_j) - ∂U^LJ/∂z_nj        Fzi = Fzi  - ∂U^LJ/∂z_i
   END DO
    Fx(i) = Fxi ; Fy(i) = Fyi ; Fz(i) = Fzi
END DO
!$OMP ENDDO
```

➔ Ici , seulement $n_j \neq i$ dans $l_i$ les pairs $(i,n_j)$ sont calculées deux fois.

➔ Pas efficace (pénalité de 100 %)

# ISM8 – TERME DE LENNARD JONES ET OPENMP

Le vecteur des forces devient spécifique à chaque thread : **Fxyz(1:Natom,1:Nthreads)**

```
!$OMP REDUCTION(+ : U^LJ)
!$OMP DO
DO i = 1,N
   …
  DO j = 1,n_i
     n_j = l_i(j)
     R_ij^2 = (x(i)-x(n_j))^2 +  (y(i)-y(n_j))^2 + (z(i)-z(n_j))^2
     ….
     Calcul de l'énergie U^LJ
     Calcul des forces
     Fx(n_j,thread) = Fx(n_j,thread) - ∂U^LJ/∂x_nj
     Fy(n_j,thread) = Fy(n_j,thread) - ∂U^LJ/∂y_nj
     Fz(n_j,thread) = Fz(n_j,thread) - ∂U^LJ/∂z_nj
  END DO
   …
END DO
!$OMP ENDDO
```

Il faut alors effectuer une réduction sur les vecteurs Fxyz(:,thread) pour obtenir le vecteur force totale sur chaque atome après le calcul du terme de Lennard Jones.

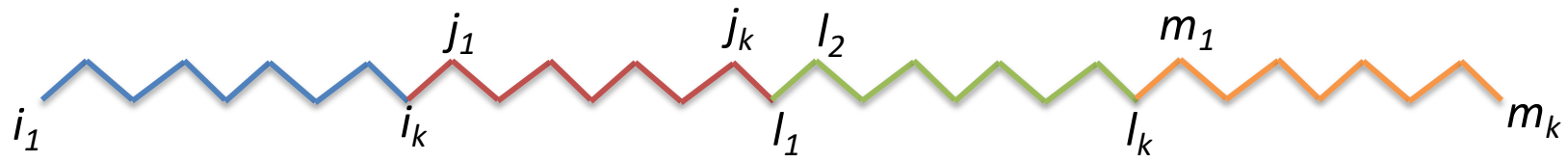
Remarque : nombre de proches voisins, en général de l'ordre de 100 à 500 => réaliser la réduction précédente ne présente pas un coût important sur des processeurs avec peu de cœurs…

…mais aujourd'hui les processeurs AMD ont 64, 96 voire 128 cœurs et les futurs processeurs ARM pour le HPC, 256 cœurs (!). Cela a revient à calculer deux fois le terme $U^{LJ}$, autrement dit il est préférable de considérer tous les voisins lors de la construction des listes de Verlet et de revenir au slide précédent.

**Conclusion : aujourd'hui il faut considérer un code avec les différentes options qui seront choisies en fonction du système microscopique et du processeur lors de l'exécution.**

# ISM8 – REMARQUE TERME DE STRETCHING ET OPENMP

Soit un système de *N* atomes qui définissent une chaine de liaisons covalentes identiques




Chaque liaison est décrite par le terme $u = k\left(r_{\alpha\beta} - r^0\right)^2$, l'énergie de « stretch » totale est donc :

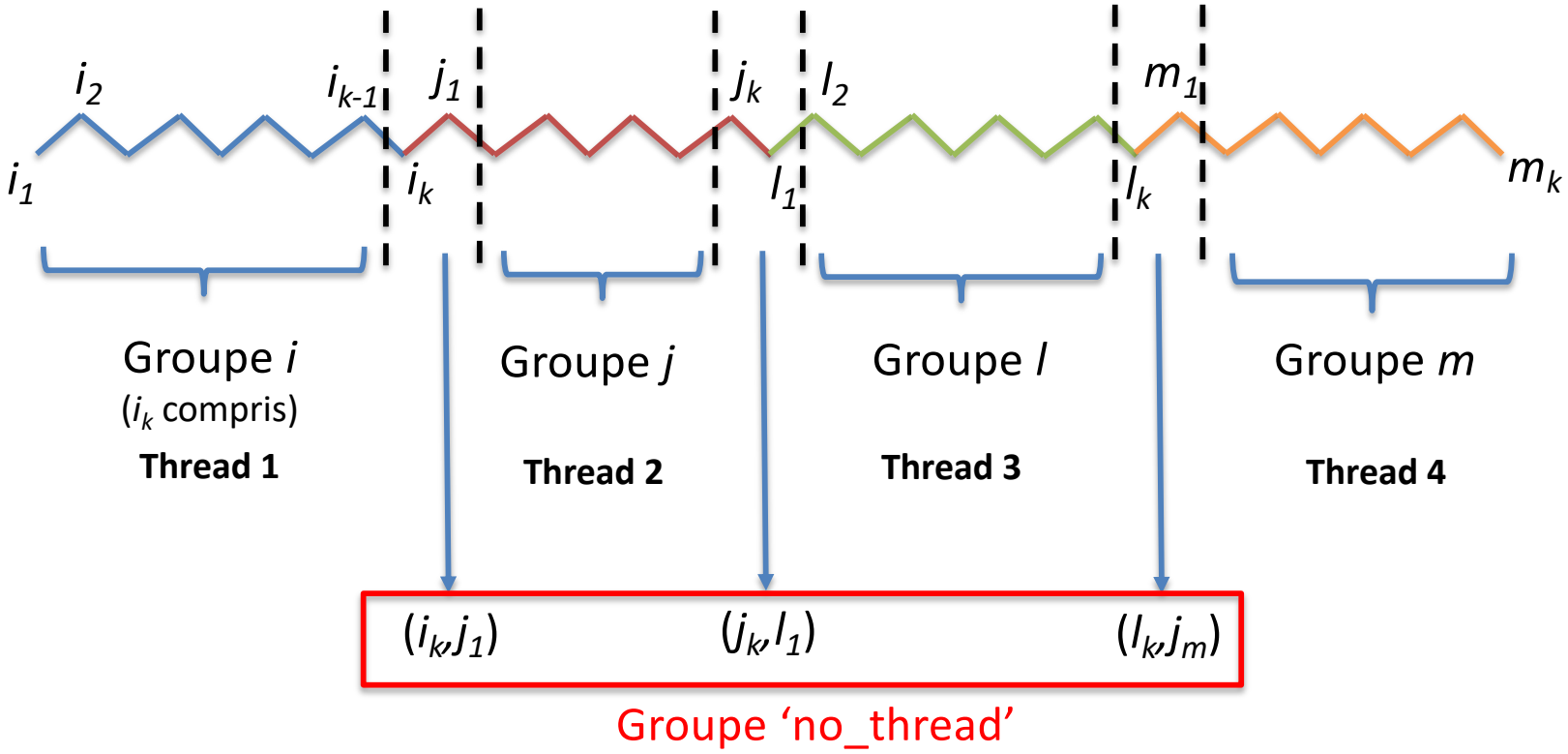
$$U^{stretch} = \sum_{(\alpha,\beta)} k\left(r_{\alpha\beta} - r^0\right)^2$$

Pour la grande majorité des liaisons covalentes : pas de problèmes de type 'race condition' car chaque liaison est définie par une paire d'atomes unique et chaque atome appartient au plus à deux liaisons, par exemple les atomes $l_1$ ou $m_1$.

Il est donc possible de proposer un algorithme efficace sur la base de threads sans directive OPENMP dans la zone « OMP PARALLEL » mais nécessitant de regrouper les liaisons covalentes par groupe de liaisons indépendantes.

# ISM8 – TERME DE STRETCHING ET OPENMP

Définition des groupes de liaison



# ISM8 – TERME DE LENNARD JONES ET VECTORISATION

Les directives de vectorisation s'appliquent à la deuxième boucle du calcul du terme de Lennard Jones (quelque soit sa complexité)

```
!$OMP DO
DO i = 1,N
!DIR$ SIMD REDUCTION (+ : U^LJ et éventuellement forces associées)
  DO j = 1,n_i
      n_j = l_i(j)
      R_ij^2 = (x(i)-x(n_j))^2 +  (y(i)-y(n_j))^2 + (z(i)-z(n_j))^2
     Calcul de l'énergie U^LJ
     Calcul des forces sur n_j
  END DO
END DO
!$OMP ENDDO
```

Pas de « race condition » avec les unités vectorielles, car un atome *i* ne peut avoir deux fois le même atome $n_j$ comme voisin.

Mais les indices $n_j$ ne sont pas à la suite => grosse pénalité lors de la lecture des données x/y/z($n_j$), => les compilateurs actuels (ifort/icc) refusent même de générer un code avec des instructions vectorielles…

# ISM8 – TERME DE LENNARD JONES ET VECTORISATION

Une stratégie « simple » consiste à décomposer la deuxième boucle en une boucle « load » et une boucle « computation » avec emploi de vecteurs locaux « thread private » (déclarés comme OMP PRIVATE). Par exemple, dans le cas de listes de Verlet étendues :

```
!$OMP PRIVATE (xlocal,ylocal,zlocal)
!$OMP DO
DO i = 1,N
  xx = x(i) ; yy = y(i) ; zz = z(i)

  DO j = 1,n_i                                                    ← load
     n_j = l_i(j)
     xlocal(j) = x(n_j) ; ylocal(j) = y(n_j) ; zlocal(j) = z(n_j) … autres paramètres des atomes n_j
  END DO

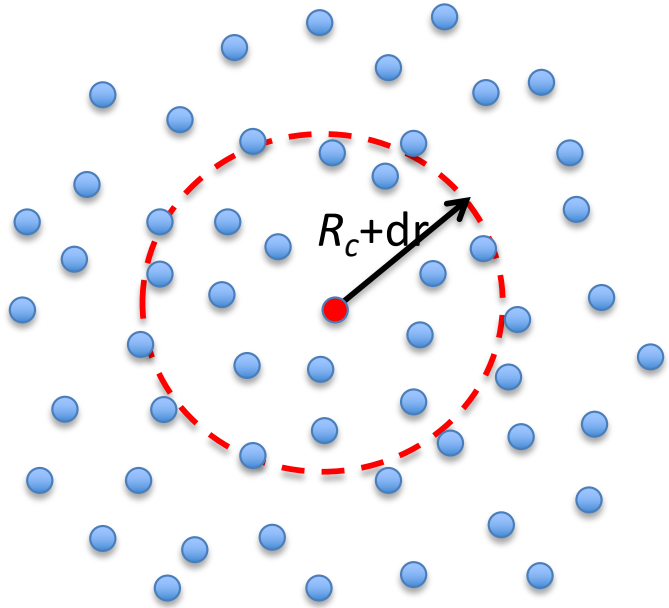
!DIR$ SIMD REDUCTION (+ : U^LJ)                                   ← computation
  DO j = 1,n_i
     R_ij^2 = (xx - xlocal(j))^2 + (yy - ylocal(j))^2 + (zz - zlocal(j))^2
     Calcul de l'énergie U^LJ
  END DO

END DO
!$OMP ENDDO
```

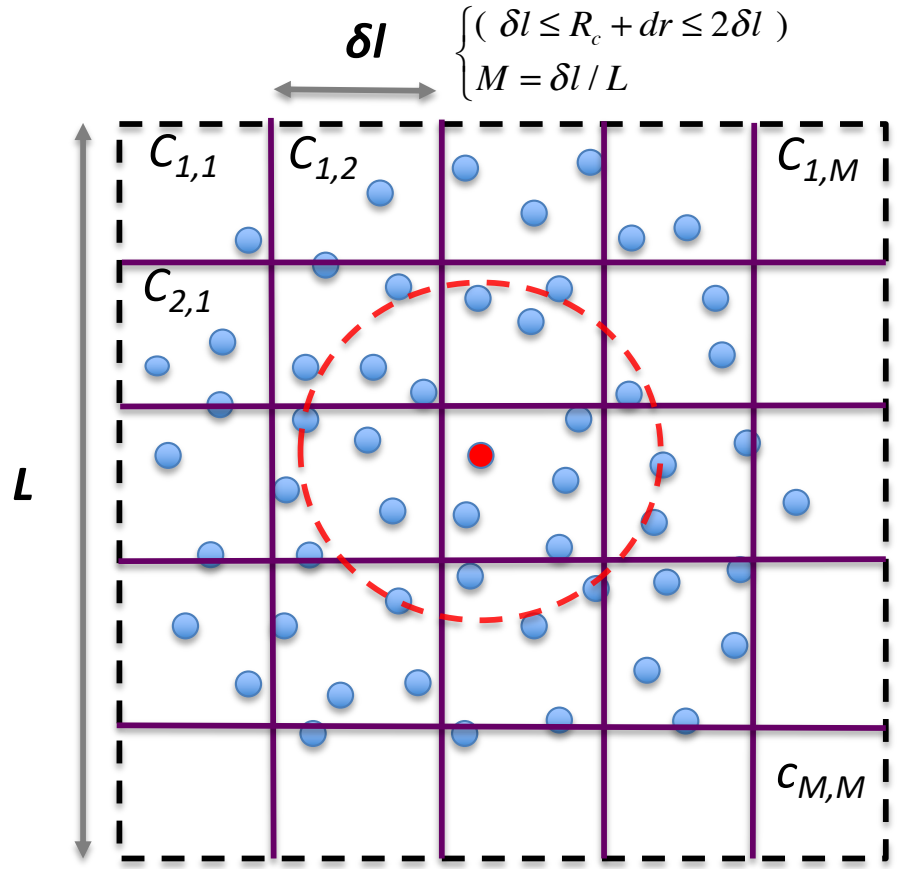
Gain proche du nombre d'unités vectorielles disponibles sur le cœur pour les calculs en réel les plus lourds (div et/ou sqrt)

# ISM8 – RECHERCHE DES ATOMES VOISINS

**Objectif : transformer une recherche globale en une recherche dans un sous domaine**

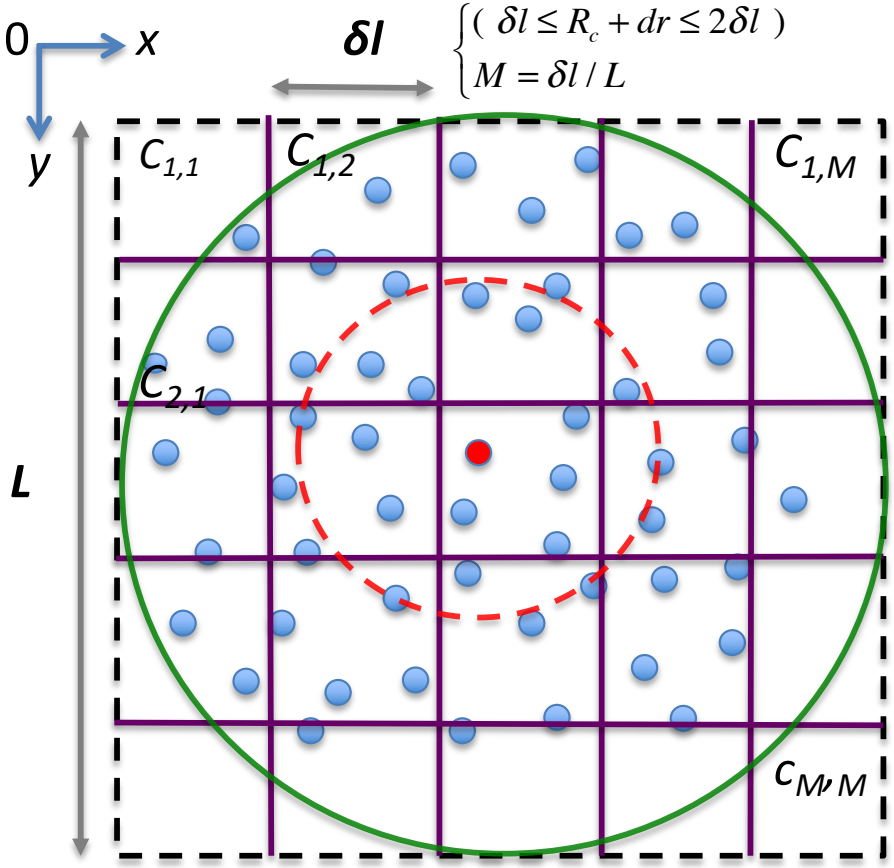



**Calcul de toutes les distances**
**algorithme en *O(N²)***




**Décomposition de domaine**
**« Divide and Conquer »**

# ISM8 – ALGORITHME DE DÉCOMPOSITION DE DOMAINE




**Principales étapes d'un algorithme en *O(N)***

1 – Pour chaque atome *A*, identifier sa cellule $c_{iA,jA}$

$$\begin{cases} i_A = \mathrm{int}(x_A / \delta l) + 1 \\ j_A = \mathrm{int}(y_A / \delta l) + 1 \end{cases} \Rightarrow \quad O(N)$$

2 – Identifier les cellules situées au plus à $2\delta l$ de $c_{iA,jA}$

$$\begin{cases} i_A - 2 \le i \le i_A + 2 \\ j_A - 2 \le j \le j_A + 2 \end{cases} \Rightarrow \quad O(N)$$

3 – Rechercher les particules à moins de $R_c+dr$ de l'atome *A* dans le jeu de cellules précédentes => *O(N)*

4 Conclusion : algorithme en *O(N)*

**Remarque : le pré-facteur de *O(N)* est ici grand, il est donc intéressant de faire une recherche des plus proches voisins de A pour $R_{long,c}=R_c+\Delta r$, *avec Δr = Kdr et de réorganiser la liste par un algorithme classique de tri => mais vous m'aviez dit que c'est du O(NlogN) ?***

# ISM8 – ALGORITHME DE DÉCOMPOSITION DE DOMAINE

**Remarque : « le pré-facteur de *O(N)* est grand, il est donc intéressant de faire une recherche des plus proches voisins de A pour $R_{long,c} = R_c + \Delta r$, *avec* $\Delta r = Kdr$ *et de réorganiser la liste par un algorithme classique de tri =>* *mais vous m'aviez dit que c'est du O(NlogN) ? »***

**Ici il ne faut pas confondre *N* (nombre total d'atome du système qui permet de définir la complexité algorithmique) et le nombre moyen $\overline{n}$ de voisins :**

**1 - $\overline{n}$ est borné (la densité de la manière n'est pas infinie);**

**2 - le tri ne s'effectue que pour ces $\overline{n}$ atomes, le temps de calcul associé est donc lui aussi borné pour un atome;**

**3 – donc pour chacun des *N* atomes du système j'effectue un tri pour une sous liste d'atomes dont la taille et donc le temps de calcul est borné…**

**Conclusion on reste bien sur une complexité algorithmique globale en *O(N)***

# ISM8 – TERME DE LENNARD JONES EN *O(N)* POUR LA DYNAMIQUE MOLÉCULAIRE

CALL decomposition_de_domaine($R_c$,$\Delta r$)

*K=dr/Δr* fois

```
!$OMP PRIVATE (xlocal,ylocal,zlocal)
!$OMP DO
DO i = 1,N
  xx = x(i) ; yy = y(i) ; zz = z(i)

  DO j = 1,n_i
     n_j = l_i(j)
     xlocal(j) = x(n_j) ; ylocal(j) = y(n_j) ; zlocal(j) = z(n_j)
  END DO

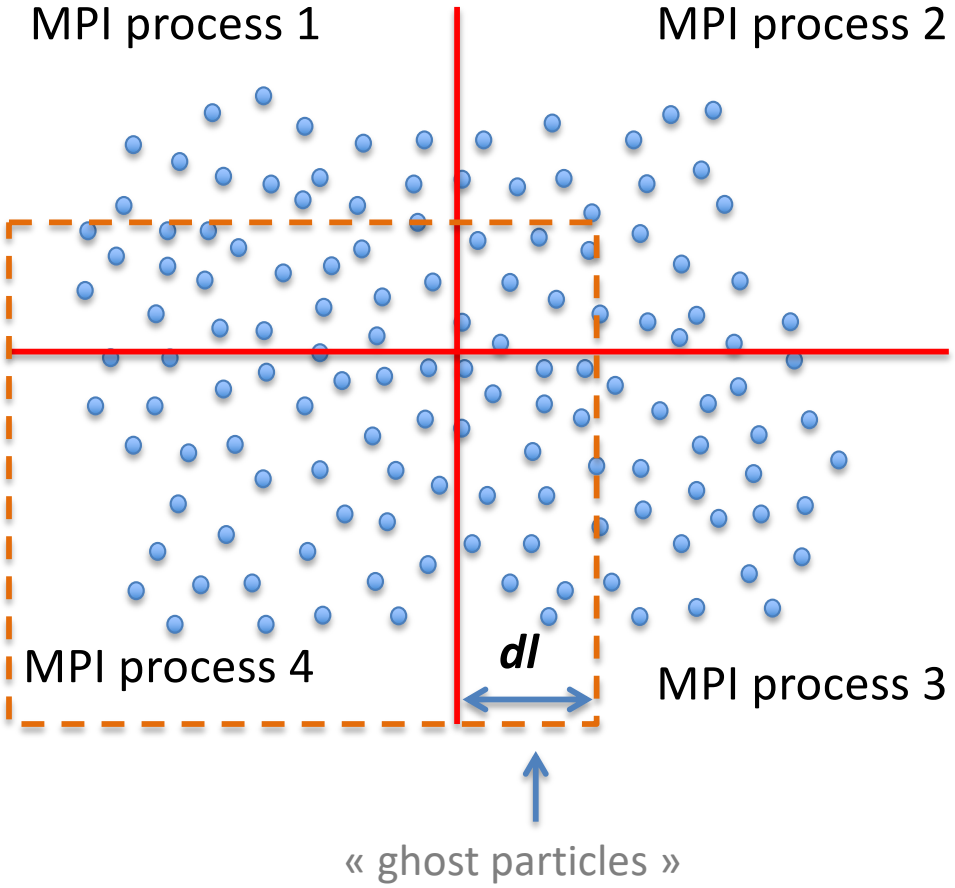
!DIR$ SIMD REDUCTION (+ : U^LJ)
  DO j = 1,n_i
     R_ij^2 = (xx - xlocal(j))^2 +  (yy - ylocal(j))^2 + (zz - zlocal(j))^2
     Calcul de l'énergie U^LJ
  END DO

END DO
!$OMP ENDDO
```

CALL résolution_equations_de_Newton(x,y,z,gx,gy,gz)

# ISM8 – DÉCOMPOSITION DE DOMAINE ET PARALLÉLISME MPI

En OPENMP chaque cœur CPU voit le même état de la mémoire, en MPI, il faut mettre à jour la mémoire de chaque jeu de CPUs regrouper sur un « nœud ». Conséquence sur l'algorithme de décomposition de domaine : il faut considérer une liste de particules « fantômes » sur numéraires pour chaque tâche MPI.




**Principales étapes d'un algorithme MPI**

1 – Répartir les particules entre les tâches MPI en fonction de leurs coordonnées

2 – Echanger les positions de particules fantômes entre les différentes tâches MPI

$$dl = R_c + \Delta r$$

3 – Appliquer à chaque sous domaine MPI, l'algorithme *O(N)* de recherche de plus proches voisins.

**Remarque** : pour assurer une bonne répartition de la charge de calcul entre les tâches MPI, il faut soit que la densité de particules soit constante en tout point, soit moduler la taille des domaines géométriques correspondant à chaque tâche MPI.

# ISM8 – TERME DE LENNARD JONES *O(N)* EN VERSION MPI

CALL **MPI**_répartition_echange_particule($R_c+\Delta r$)

CALL decomposition_de_domaine($R_c+\Delta r$)

*K=dr/Δr* fois

```
!$OMP PRIVATE (xlocal,ylocal,zlocal)
!$OMP DO
DO i = 1,N
  xx = x(i) ; yy = y(i) ; zz = z(i)

  DO j = 1,n_i   ⟶ en MPI, n_i est potentiellement plus grand => surcoût
     n_j = l_i(j)
     xlocal(j) = x(n_j) ; ylocal(j) = y(n_j) ; zlocal(j) = z(n_j)
  END DO

!DIR$ SIMD REDUCTION (+ : U^LJ)
  DO j = 1,n_i
     R_ij^2 = (xx - xlocal(j))^2 + (yy - ylocal(j))^2 + (zz - zlocal(j))^2
     Calcul de l'énergie U^LJ
  END DO

END DO
!$OMP ENDDO
```

CALL résolution_equations_de_Newton()

**La liste de particules fantômes évite de faire une mise à jour MPI des forces !**

# ISM8 – DERNIÈRES REMARQUES

1 - Cet algorithme reste valable quelque soit le terme d'interaction « à courte portée » considéré, *i.e.* lorsque l'on considère un terme d'interaction par paire à qui on applique une troncature.

2 - Chaque tâche MPI correspond à un domaine d'espace. Afin de minimiser le coût des communications MPI, ile faut faire correspondre la topologie des tâches MPI à celle des domaines d'espace.

3 – Aujourd'hui, on arrive à pousser l'efficacité de ce type d'algorithme à 250 particules/atomes par cœur de calcul. Pour aller au delà il faudra significativement améliorer les performances des réseaux d'inter-connection.

4 – Toutefois, quoique l'on fasse, on réalise au mieux des simulations à l'échelle de quelques dizaines de nanoseconde par jour sur une architecture de calcul de type CPU, et cette valeur ne va pas évoluer de manière spectaculaire dans un avenir proche…

5 - La machine ANTON est un processeur dédié de 512 cœurs dans sa version actuelle, avec une mémoire physique commune à tous.

# ISM8 – EXERCICE : ATTENUATION D'UNE COUPURE

Soit le polynôme d'ordre 5 suivant

$$P_5(x) = 1 - 10\left(\frac{x - x_{\min}}{x_{\max} - x_{\min}}\right)^3 + 15\left(\frac{x - x_{\min}}{x_{\max} - x_{\min}}\right)^4 - 6\left(\frac{x - x_{\min}}{x_{\max} - x_{\min}}\right)^5$$

$$\text{avec} \quad x_{\min} \leq x \leq x_{\max}$$

1 – Quelles sont ses valeurs et celles de sa dérivée première en $x_{min}$ et $x_{max}$ ?

2 – Comment employer ce polynôme pour atténuer l'application d'un rayon de coupure à une interaction de type Coulombienne ou de Lennard-Jones ?

3 – Comment réécrire les dérivées d'une fonction f($x$) à qui on applique le polynôme $P_5(x)$ ?

# ISM8 – ANNALE EXAMEN 2019

**I.10 Comment se comporte la fonction de distribution radiale *g*(*r*) quand $r \to +\infty$ pour un système condensé homogène ?**

Réponse : une ligne maximum

**I.11 Comment mettre à profit cette propriété de la fonction *g*(*r*) pour définir l'ordre de grandeur de la taille d'un système périodique (ici une boite cubique de dimension *L*) permettant de simuler le système condensé précédent ?**

Réponse : deux lignes maximum

**I.22 La complexité algorithmique de la méthode de recherche des plus proches voisins basé sur une décomposition de domaine est *O(N)* ou *N* est la taille (en nombre de particule) de la boite simulée avec des conditions périodiques. Une telle méthode se parallélise efficacement en MPI. Que faut-il veiller à vérifier pour que cette efficacité en MPI soit maximum ?**

Réponses : la dimension géométrique des domaines doit être identique, ces domaines doivent contenir le même nombre de particule, le système microscopique présente une interface, le système microscopique est homogène

**I.38 Soit une fonction énergie potentielle de la forme $U = \sum_{i=1}^{N} \sum_{j>i}^{N} f(r_{ij})$. Un schéma de parallélisation de type OPENMP bien adapté à cette double somme consiste**

Réponses : à ajouter une directive OPENMP de type !$OMP DO au niveau de la deuxième boucle du code correspondant à la deuxième somme de *U*, à ajouter une directive OPENMP de type !$OMP SINGLE au niveau de la première boucle du code correspondant à la première somme de *U*, à ajouter une directive OPENMP de type !$OMP DO au niveau de la première boucle du code correspondant à la première somme de *U*

**I.39 Soit un système microscopique composé de *N* particules et dont la fonction énergie potentielle est de la forme $U = \sum_{i=1}^{N} \sum_{j>1}^{N} f(r_{ij})$ où $f$ n'est jamais négligeable quelle que soit la distance $r_{ij}$. Une particule *i* de ce système**

Réponses : peut interagir avec elle-même, ne peut jamais interagir simultanément deux fois avec la même particule *j*, peut interagir avec aucune particule

**I.40 Soit un système microscopique composé de *N* particules et dont le fonction énergie potentielle est de la forme $U = \sum_{i=1}^{N} \sum_{i \neq j, j=1}^{N} \sum_{k \neq (i,j), k=1}^{N} f(r_{ij}, r_{jk}, r_{ik})$. Peut-on appliquer un directive de vectorisation de type !DIR SIMD à la 3ième boucle d'un code permettant de calculer cette fonction ?**

Réponses : oui, non, parfois

# ISM8 – CODE DE SIMULATION : INTRODUCTION DES LISTES DE VERLET EN $O(N^2)$

Introduire dans le code de simulation un tableau à deux dimensions :

*neighbor* ( 1:N_particule , 0:**2***n_max_neighbor)

Ici, n_max_neighbor est une estimation du nombre maximum de plus proche voisin d'une particule pour un rayon de coupure $R_c$. On peut l'estimer suivant la formule :

density = N_particule / (Lx*Ly*Lz) ; n_max_density = int(density*4*$\pi$*$R_c^3$) **x2**

Lx/Ly/Lz sont les dimensions de la boite périodique, le facteur **x2** permet de prendre en compte des fluctuations importantes à la densité moyenne 'density'. L'algorithme de construction des listes de Verlet en $O(N^2)$ pour des conditions périodiques sera de la forme :

```
DO i = 1,N_particule
    ni = neighbor(i,0)
    DO j = i+1,N_particule
        DO i_sym = 1,27
            Xj_loc = x(j) + coordonnée X du i_sym^ième vecteur {η}
            Yj_loc = y(j) + coordonnée Y du i_sym^ième vecteur {η}
            Zj_loc = z(j) + coordonnée Z du i_sym^ième vecteur {η}
            R_ij^2 = (x(i)- Xj_loc )^2 +  (y(i)- Yj_loc )^2 + (z(i)- Zj_loc )^2
            if (R_ij^2 < R_c*R_c)  ! mise en mémoire de la paire [i,j(i_sym)]
              ni = ni +1 ; mi = 2*ni -1  ; neighbor(i,mi) = j ; neighbor(i,mi+1) = i_sym
            end if
        END DO
    END DO
END DO
```